Optimizing Launch Latency for Confidential VMs with Device Passthrough in the Linux KVM Hypervisor

Hao-Jung Wei
National Taiwan University
Taipei City, Taiwan
r12922072@csie.ntu.edu.tw

Shih-Wei Li
National Taiwan University
Taipei City, Taiwan
shihwei@csie.ntu.edu.tw

Abstract

Confidential Virtual Machines (CVMs) provide a secure execution environment for sensitive cloud workloads. However, when hosted on commodity Linux KVM hypervisors, their use of high-performance passthrough devices via the VFIO driver introduces significant latency at launch because Virtual Function I/O (VFIO) must allocate and map the entire CVM memory to the I/O Memory Management Unit (IOMMU). In this work, we analyze the start-up overhead of CVMs with device passthrough on a predominant KVM-based CVM implementation with AMD SEV-SNP. We propose an optimization, Targeted VFIO Mapping, which leverages the insight that mapping only the bounce buffers to the IOMMU—rather than the entire guest memory—is sufficient to support functional and secure device passthrough, while greatly reducing start-up latency. Our evaluation demonstrates that Targeted VFIO Mapping significantly streamlines initialization, yielding up to a 55% reduction in start-up time of Linux-based CVMs on KVM.

CCS Concepts: • **Security and privacy → Virtualization and security**.

Keywords: Confidential Computing, Device Passthrough

ACM Reference Format:
Hao-Jung Wei and Shih-Wei Li. 2026. Optimizing Launch Latency for Confidential VMs with Device Passthrough in the Linux KVM Hypervisor. In *9th Workshop on System Software for Trusted Execution (SysTEX '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3805690.3805730

1 Introduction

Confidential Virtual Machines (CVMs) streamline the deployment of sensitive workloads by offering high compatibility with existing applications. By integrating hardware and software extensions into the virtualization stack, CVM technology isolates workloads from untrusted hosts and physical devices. This allows users to leverage standard hypervisors to spawn secure environments, protecting data and code even within potentially compromised infrastructure. Various CVM solutions have been introduced for the mainstream x86 and Arm architectures.

To achieve optimal performance, CVMs employ device passthrough, which provides near-host performance by granting them direct access to physical devices. Those directly exposed to CVMs, like network devices or accelerators (e.g., GPUs or FPGA), can use direct memory access (DMA) to access CVM memory. As a result, virtualization overhead is significantly reduced by eliminating hypervisor mediation, which entails additional data copying, notification latency, and duplication of software abstractions, such as device drivers and the network stack. In Linux KVM, device passthrough to VMs is primarily facilitated by the Virtual Function I/O (VFIO) [28] driver. VFIO provides a secure, device-agnostic interface that allows the host to manage device resources. VFIO maps the device to the VM's address space. VFIO leverages the Input/Output Memory Management Unit (IOMMU) to restrict device access boundaries, ensuring that the assigned device can only access its own memory and not the memory allocated to other VMs or the host. The software, such as the OS kernel or hypervisor, manages an IOMMU page table for each attached device to enforce memory access controls.

In our work, we evaluated predominant CVM implementations, namely AMD SEV-SNP VMs [1] with device passthrough over VFIO on KVM, to assess their impact on startup latency. We find that for SEV-SNP VMs with multiple GBs of RAM, the high start-up time overhead is significant. Specifically, in the worst case, the boot times of CVMs using device passthrough are more than 2x compared to those of CVMs using paravirtualized virtio [29]. Further analysis indicates that the overhead originates from VFIO, which maps the entire CVM memory to the IOMMU during CVM launch, thereby incurring substantial latency due to page table mapping and memory allocation. As CVMs that leverage passthrough devices, such as GPUs, in training pipelines are increasingly configured with terabyte-scale memory to serve confidential AI workloads in practice, there is an urgent need to optimize CVM startup latency when using device passthrough.

This work is licensed under a Creative Commons Attribution 4.0 International License.
SysTEX '26, Edinburgh, Scotland Uk
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2607-1/2026/04
https://doi.org/10.1145/3805690.3805730

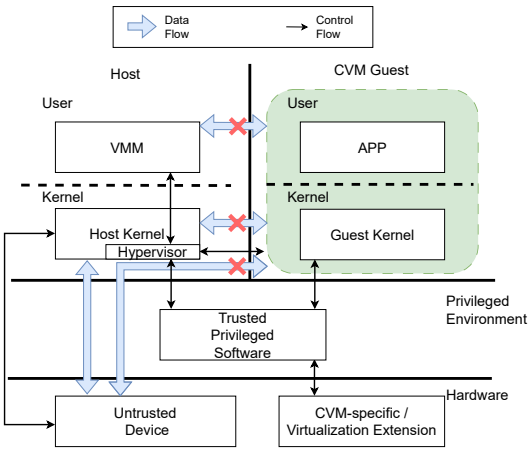

Figure 1. The architecture of CVM systems.

We make a novel observation that, although the entire memory is mapped to the IOMMU, to enforce memory protection, operating systems such as Linux restrict device access to exclusively be proxied over the bounce buffer, a small pre-allocated memory region, rather than to all memory when executing in CVM environments. Bounce buffering is a crucial data-sharing mechanism between CVMs and I/O devices that enables the latter to access a small portion of unencrypted CVM memory. The use of bounce buffering in CVMs offers an opportunity to optimize boot-time overhead in device passthrough using VFIO. We exploit the observation and propose a **Targeted VFIO Mapping** optimization. Instead of mapping the entire guest memory, we can map the bounce buffer to the IOMMU. By eliminating redundant page-table operations and boot-time memory-allocation overhead, we significantly reduce the launch time of CVMs with device passthrough using the de facto VFIO framework on commodity KVM, without compromising functionality or security.

We prototyped the Targeted VFIO Mapping optimization by extending the commodity KVM-based virtualization framework. Our evaluation of widely used AMD SEV-SNP hardware shows that, on Linux-based SEV-SNP CVMs with 64GB of memory, our Targeted VFIO Mapping approach reduces launch time by 55% relative to the default VFIO configuration, delivering performance gains while fully preserving the robust security guarantees of bounce buffering. The optimization approach facilitates CVM use cases in KVM that require minimal start-up latency while benefiting from the efficiency of device passthrough.

2 Background

2.1 Confidential Virtual Machines (CVMs)

Figure 1 illustrates the architecture of CVM systems. It consists of a userspace virtual machine monitor (VMM), the hypervisor, and the host OS kernel, trusted privileged software, and a guest CVM. The VMM (e.g., QEMU [24], crosvm [13]) works with the hypervisor (e.g., KVM in the Linux kernel) to create and manage VMs. The VMM and the hypervisor provide the most complex functionality, including scheduling and memory allocation. CVM systems rely on trusted software components, such as the pKVM hypervisor in the pKVM [12] framework in Linux, the firmware in AMD SEV-SNP [1], and the realm management monitor (RMM) in Arm CCA [10], to protect the confidentiality and integrity of CVMs against the host. These components serve as the trusted computing base (TCB) that executes in a higher-privileged environment to enforce protection. In CVM systems with hardware extensions (AMD SEV-SNP [1], Intel TDX [2], Arm CCA [10]), the TCB leverages these extensions to provide protection. In CVM systems without dedicated hardware extensions (e.g., pKVM [12] and SeKVM [18–20]), the TCB leverages existing hardware virtualization extensions to protect CVMs against untrusted entities.

Typically, memory protection is enforced by hardware mechanisms that vary by architecture. This is typically enforced through hardware mechanisms that check against a secure table structure (e.g., Arm CCA's Granule Protection Table (GPT), SeKVM and pKVM's Stage-2 Page Table (S2PT), or AMD SEV-SNP's Reverse Map (RMP) Table [1]), memory encryption ([1, 2, 9]), or a hybrid of both. Untrusted entities, e.g., untrusted devices and host software, are protected against tampering with a CVM's private memory, which, when hardware extensions for CVMs are incorporated, is encrypted by default[1]. A per-guest private key is associated with each CVM to encrypt memory. Conversely, shared memory remains accessible to untrusted entities, thereby facilitating I/O and communication. The shared regions are unencrypted memory explicitly designated by the CVM, enabling secure data exchange via defined protocols.

2.2 IOMMU and Device Memory Access

IOMMU is a critical component in memory translation. DMA requests issued from the device are subject to the IOMMU, which translates the input I/O virtual address (IOVA) into a host physical address (HPA). IOVA translation enables safe DMA operations by ensuring that physical devices can only access authorized regions of memory. IOMMU implementations either support a single-stage or two-stage memory translation. Arm provides an IOMMU denoted System Memory Management Unit (SMMU).

Device passthrough is way to achieve high-performance I/O virtualization. The goal is to allow the guest VM to directly utilize the physical device. The aim is to eliminate the virtualization overhead, such as data copy, notification latency, and duplication of the network stack, required for

[1]pKVM and SeKVM utilize S2PT for memory isolation; their CVMs' memory is unencrypted.

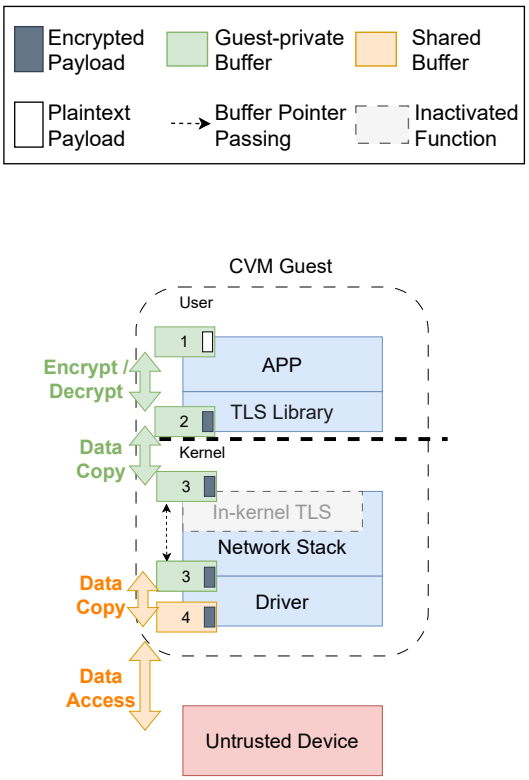

Figure 2. Bounce Buffering for Network Device of CVM

paravirtualization or emulation approaches, and deliver near-native performance. In KVM, to enable data transfer between the guest VM and the device, the VMM (e.g., QEMU) must map the guest VM's memory to the IOMMU via the VFIO interface, allowing the guest to communicate with the device using Guest Physical Addresses (GPA). Without the GPA-to-HPA mapping provided by the IOMMU, the guest cannot program the device because the device recognizes only HPA. By restricting guest devices' access to GPAs, this also prevents them from performing malicious DMA to the host's memory or to other VMs' memory.

Bounce Buffering. Traditionally, OSes employ bounce buffering to circumvent hardware addressing limitations, allocating a staging memory area within the devices' accessible range. Data is then copied, or bounced, between this reachable bounce buffer and the original memory location in the kernel space to facilitate the transfer. Bounce buffering is later adopted as a secure enhancement to restrict the device's DMA capabilities to the designated buffer area.

In the CVM paradigm, bounce buffering is repurposed as a fundamental security boundary to facilitate I/O while maintaining isolation. By default, the host is cryptographically or architecturally barred from accessing a CVM's private memory. An intuitive approach to resolve this functional paradox without disabling memory protection is for CVMs to switch between private and shared memory at runtime. However, this dynamic change requires non-trivial overhead, stemming from extra memory copying, VM exits, micro-architecture state maintenance, and inter-processor communication [16]. Thus, instead of adopting runtime sharing of arbitrary private memory, CVMs generally reserve a dedicated "bounce buffer" region at boot to serve as a bridge between trust domains. In this context, CVMs explicitly disable memory protection of pages allocated for the shared bounce buffers—such as encryption or hardware access controls—to enable unconstrained host and device access.

This mechanism is particularly critical on platforms that lack end-to-end I/O protection (e.g., AMD SEV-TIO [4]), which would otherwise allow devices to operate directly on ciphertexts. In the absence of such hardware, CVMs employ bounce buffering not only for physical device passthrough but also for paravirtualized virtio devices. As illustrated in Figure 2 for a virtual network device use case, an untrusted host virtual device or hardware peripheral can only interact with the shared bounce buffer (marked in orange), ensuring the remainder of the guest's private memory remains protected. CVMs strictly confine device access to dedicated bounce buffers, thereby maintaining isolation for the remainder of the private memory.

For outbound transfers, the CVM copies data from its private memory into the shared bounce buffer, effectively exporting it to the untrusted host. For inbound transfers, the device writes to the shared buffer, after which the guest performs a validated copy into its private space. By enforcing this intermediate staging, the CVM ensures that untrusted entities interact only with transient data copies, which it protects, thereby shielding the confidentiality and integrity of its private memory from unauthorized access.

3 Threat Model & Assumptions

We assume a standard confidential computing trust model in which the CVM's own software stack (i.e., application and guest kernel), the confidential computing hardware, and the secure software components are trusted. The host software (hypervisor, host OS kernel, and VMM), host devices, and co-resident VMs are considered malicious. We assume that these entities will attempt to compromise the confidentiality and integrity of the CVM's resources, including its private CPU states and memory. Although a CVM relies on untrusted entities for I/O transport, we assume it employs standard end-to-end cryptographic protocols to protect payloads. We also assume that CVM users do not voluntarily expose their sensitive data. Denial-of-service, interrupt-related [26], and side-channel attacks [17], [11] are also out of our scope.

4 Evaluation

To understand the start-up delay incurred by VFIO, we evaluate the start-up time of CVMs on the predominant SEV-SNP hardware. Here, we define the CVM start-up time as the time from CVM launch until the CVM outputs the first line. We use an Expect script [21] to measure the start-up time. Specifically, the script launches the VM, monitors its console output, and records the time of the first message that appears from the VM's serial output. In an SEV-SNP VM, this measures the time from initialization until the guest firmware, OVMF [8], starts running. We run the test 5 times and take the average of the measured times.

Configuration. We run the AMD SEV-SNP CVMs using QEMU on a Dell PowerEdge R7515 server with an AMD EPYC 7543P Zen3 3.5 GHz processor. The host and guest Linux kernel [7], OVMF [8], and QEMU [6] are all built from the AMDESE repositories, using branches snp-host-latest, snp-guest-latest, snp-latest, and snp-latest, respectively. The OS of the host machines and VMs is Ubuntu 22.04. Each VM is spawned with 8 cores. We test SEV-SNP VMs with varying memory sizes across two configurations (8 and 64 GB) and use two profiles: network device passthrough via VFIO and virtio networking [29]. In our evaluation, we use a simplified setup in which the entire network device is assigned to the SEV-SNP VMs. Note that, in practice, VFIO is often used to map the virtual function (VF) of a device with Single Root I/O Virtualization (SR-IOV) capability to VMs.

Table 1. Evaluation of SEV-SNP VM Start-up Times. The Original column shows the times without optimization. The Optimized column shows the performance with Targeted VFIO Mapping. The Improvement column shows the performance improvement over the original setup.

Config	Original	Optimized	Improvement
SEV-SNP 8GB w/ VFIO	2.63 sec	1.94 sec	26%
SEV-SNP 64GB w/ VFIO	10.41 sec	4.73 sec	55%
SEV-SNP 8GB w/ virtio	1.71 sec		
SEV-SNP 64GB w/ virtio	4.5 sec		

Evaluation Result & Discussion. Table 1 presents our evaluation results. We first find that SEV-SNP VMs with virtio incur substantially higher overhead than unprotected VMs on KVM, which exhibit constant low overhead (startup time less than 0.2 seconds). We find that, when creating an SEV-SNP VM, this overhead stems from QEMU issuing the ioctl system call *KVM_SET_MEMORY_ATTRIBUTES* to update per-page metadata during CVM initialization, and it scales with the amount of memory allocated to the CVM. We leave an optimization to reduce the system call overhead for future work.

This system call overhead persists in SEV-VMs with VFIO. However, we find that VFIO incurs significant overhead to the start-up time of SEV-SNP VMs compared to virtio. The two CVM profiles with VFIO in Table 1 perform substantially worse than the virtio counterparts with the same memory size – the VFIO CVM profiles are roughly 1 and 6 seconds slower than the virtio profiles, each for 8 and 64 GB size. This performance penalty stems from the requirement to facilitate secure device access via VFIO, and can be attributed to the following main factors specifically:

- **IOMMU Mapping Latency.** Before executing the CVM, the host's VFIO framework from the VMM, e.g., QEMU, must map the entire guest memory to the IOMMU to establish GPA-to-HPA mappings to enable device passthrough. Because IOMMU translations typically operate at a 4KB granularity, this process scales poorly with memory size. For instance, mapping an 8GB VM requires approximately 2,000,000 individual page table updates, making this the primary source of boot-time delay.
- **Eager Page Allocation and Initialization.** While KVM traditionally employs demand paging, the initialization of VFIO-mapped regions forces eager page allocation. VFIO clears the contents of these pages before establishing mappings, thereby triggering a large volume of page faults that must be handled during the boot sequence. This results in substantial launch-time delays due to page allocation and initialization, which are absent in standard, non-passthrough VM configurations, in which faults occur sporadically during VM execution.

5 Optimization: Targeted VFIO Mapping

5.1 Design

The VFIO framework requires an exhaustive mapping of the VM's guest physical address space to the IOMMU during VM launch. This requirement arises because the host-side VFIO lacks *a priori* knowledge of the specific memory regions the guest operating system will designate for DMA. To ensure operational stability across arbitrary guest kernel configurations and DMA buffer allocations, VFIO must adopt a conservative stance by pinning and mapping the guest's entire memory footprint. By establishing these mappings in advance, the system ensures that any device-issued DMA request can be successfully translated by the IOMMU, thereby precluding runtime translation faults or fatal I/O errors that would result from unmapped address ranges. This approach ensures functionality but incurs significant performance overhead, as demonstrated in Table 1.

VFIO's legacy "map-everything" approach is fundamentally incompatible with the architecture of confidential computing. In a CVM environment, all peripheral I/O is restricted to bounce buffers that constitute only a small fraction of the overall guest memory. Bounce buffers are unencrypted memory that is explicitly shared with the host during the guest kernel's boot and serve as a staging shared-memory communication channel for data transferring. The key insight we

made in this work is that pinning and mapping gigabytes of encrypted, private memory is an expensive and redundant operation that unnecessarily inflates VM startup latency.

Based on this insight, we propose **Targeted VFIO Mapping**, an optimization that streamlines VFIO initialization. Rather than performing a blanket mapping of the entire guest physical address space, our design uses a paravirtualization approach that intercepts the memory allocation phase and restricts IOMMU programming strictly to the address ranges designated for bounce buffers. This approach is feasible as modern guest operating systems, such as Linux, reserve static physical memory regions at fixed addresses for bounce buffers early in the boot process. By modifying the VFIO framework to bypass the pinning of encrypted memory blocks—which are never targets of legitimate DMA—we eliminate the overhead of locking the guest's entire address space. Instead, VFIO is optimized to construct IOMMU page tables that map only the CVM's shared bounce buffer memory, aligning the host's I/O configuration with the CVM's actual security boundaries.

5.2 Evaluation

Proof-of-concept Implementation. We implement a PoC based on the proposed Targeted VFIO mapping design to reduce the number of IOMMU mapping requests to accelerate CVM boot time. Instead of mapping the entire guest memory, we modify the VMM to selectively map the guest Linux's bounce buffer to the IOMMU page table of the passthrough device. Typically, on Linux, the total size of the bounce buffer is at most 512 MB. Compared to mapping the entire VM memory, solely mapping the bounce buffer is significantly more efficient.

We implement the PoC for CVMs running a guest Linux kernel on the mainline KVM for AMD SEV-SNP by extending QEMU. We first identify the location of the bounce buffer in the CVMs' guest physical memory. On SEV-SNP, the guest Linux allocates the bounce buffer at startup from a known fixed address. We suppress IOMMU mapping requests for every memory region except the bounce buffers by validating the target address range of the requests in QEMU. In detail, we inspect the arguments of the invocations of the system call `ioctl(..., VFIO_IOMMU_MAP_DMA, ...)`. In each invocation, the VMM provides the mapping size, the GPA, and the HVA. The HVA is then translated into the corresponding HPA by VFIO to establish the GPA-to-HPA mappings. We validate whether the requested memory range, which is indicated by the GPA and the mapping size, falls within the bounce buffer regions. To prototype the effect of Targeted VFIO Mapping, we discard any requests that do not target the bounce buffer. In our prototype, we do not encounter cases where the bounce buffer is partially overlapped by the requested memory range.

Performance Evaluation. We evaluate the start-up time with the Targeted VFIO Mapping PoC with the same configuration and experimental setup as described in Section 4. We observe that the SEV-SNP VMs statically allocate a 512 MB bounce buffer, independent of the total available system memory. Consequently, we configure the IOMMU to map this 512 MB bounce buffer.

The evaluation results are presented in Table 1. The improvement is computed as the ratio of the decrease in start-up time to the original start-up time. The results show that our optimization significantly improves the performance. For an SEV-SNP VM with 8 and 64 GB of memory, our optimization reduces start-up time by 26% and 55%, respectively, achieving performance comparable to SEV-SNP VMs with virtio while maintaining full passthrough I/O functionality.

6 Discussion

VFIO Performance of CVMs on pKVM. At present, unlike hardware-based SEV-SNP platforms, the software-based pKVM framework in Linux does not include a DMA protection scheme to restrict hardware device access to CVM memory. Recent efforts [25] have proposed extending the pKVM's TCB to manage Arm's SMMU. Under this model, the SMMU is unmapped from the host's address space, and the TCB employs a trap-and-emulate mechanism to validate host configurations and uses the SMMU's nested page tables to enforce memory isolation.

Crucially, device passthrough currently remains outside the scope of pKVM's SMMU protection and is incompatible with the VFIO framework. If this limitation is addressed, we anticipate that VFIO performance will further degrade, as the required trap-and-emulate overhead would compound existing latencies in IOMMU mapping and memory allocation. To provide a baseline for future performance expectations, we have extended a pKVM implementation (omitting host DMA protection) to support and quantify the performance of VFIO-based device passthrough.

Similar to our efforts on SEV, we implement the Targeted VFIO Mapping PoC for Linux-based CVMs on the Linux pKVM framework for Arm by modifying crosvm [13], the respective VMM. On pKVM, we retrieve the location of the bounce buffer from the device tree property `RESTRICTED_DMA` [27] from crosvm, which is exposed to the guest CVM.

We evaluated the performance of our pKVM prototype on the Ampere Altra AVA Developer Platform with Arm Neoverse-N1 1.7Ghz cores [3]. The host kernel and the guest kernel are based on the pKVM's official repo (branch pkvm-integration-6.5) [14]. Table 2 shows that for Linux-based CVMs running on KVM with pKVM support. Note that, due to hardware constraints (the AVA server has 32 GB of RAM), we tested CVM with 8 and 24 GB of memory. The results demonstrate the effectiveness of Targeted VFIO Mapping

for pKVM with a different VMM implementation, achieving performance characteristics similar to those on SEV-SNP.

Table 2. Evaluation of CVM Total Start-up Times on pKVM.

Config	Original	Optimized	Improvement
pKVM 8GB w/ VFIO	2.09 sec	1.49 sec	29%
pKVM 24GB w/ VFIO	4.49 sec	2.78 sec	38%
pKVM 8GB w/ virtio	1.27 sec		
pKVM 24GB w/ virtio	2.58 sec		

Related Work. FastIOV [22] also investigates the start-up time of device passthrough with VFIO for microVMs, such as Firecracker [5] and Kata [23], that host containers. These microVMs have much smaller memory footprints (e.g., less than 1 GB of RAM, 512 MB) than CVMs used in conventional deployments, which typically allocate multiple GBs of memory. Therefore, in contrast to our findings that show a significant slowdown in CVM launch time stemming from VFIO's memory mapping and allocation, the authors of FastIOV report that mapping memory to the IOMMU accounts for only a small portion of the total boot-time overhead. On the other hand, FastIOV focuses on general unprotected container use cases, but not for confidential computing scenarios. Furthermore, FastIOV focuses on reducing the startup time of containers hosted in microVMs, not in general CVMs. It optimizes characteristics specific to this microVM-container deployment to improve start-up time by minimizing lock contention, enabling asynchronous driver initialization, avoiding mapping microVM image memory to the IOMMU, and adopting lazy initialization for mapped memory. Conversely, our Targeted VFIO Mapping optimization addresses the scalability limitations of large-memory CVMs. By exploiting how CVMs use bounce buffers, we mitigate IOMMU mapping requests and the cost of eager page allocation, which constitute the primary overhead for guests with multi-GB RAM footprints.

Recent advancements in confidential computing, such as AMD SEV-TIO [4], provide end-to-end I/O protection, enabling devices to access encrypted CVM private memory directly. By securing the DMA path, these architectures eliminate the requirement for bounce buffering. Consequently, Targeted VFIO Mapping might appear incompatible in such contexts. However, we contend that the guest kernel in CVMs may still utilize bounce buffering as a security enhancement for intra-guest DMA protection, restricting a device's DMA capabilities to a strictly designated buffer area. Under this security model, Targeted VFIO Mapping remains applicable for reducing the launch-time latency of CVMs, even on hardware featuring end-to-end I/O protection.

Future Work. We did not provide evaluation results for Arm CCA due to the unavailability of current hardware. On the other hand, the performance of CVM with device passthrough on Intel TDX exhibits startup latency behavior comparable to that observed in this study with VFIO. We expect the proposed Targeted VFIO Mapping optimization to apply to TDX and CCA, as both architectures utilize bounce buffers and permit CVMs to explicitly share memory with the host and devices. We leave the evaluation and porting of our solution to these architectures for future work.

The end-to-end CVM boot time may be affected by factors other than VFIO. While Targeted VFIO Mapping optimizes the hypervisor-induced delays during CVM boot until the guest firmware (e.g., OVMF) starts, the total end-to-end startup time may still be dominated by firmware initialization and secure boot mechanisms [15]. In particular, on Intel TDX, the guest CVM's OVMF page handling can take up to several minutes during boot. End-to-end optimization of the CVM launch time is an area of future work.

By default, the Linux kernel employs a 4KB page granularity for IOMMU mappings (as used in our evaluation setup). However, modern IOMMUs support huge-page mappings with 2MB or 1GB granularity. Utilizing huge pages reduces the overhead of IOMMU page-table traversals when constructing memory mappings in VFIO for device passthrough. Evaluating the performance gains offered by huge-page optimization is a subject for future work.

7 Conclusion

In this work, we addressed the significant start-up latency imposed by KVM's VFIO framework for device passthrough on CVMs. Our analysis reveals that pinning and mapping an entire encrypted CVM memory footprint is an overfit, as CVM I/O is architecturally confined to small, unencrypted bounce buffers. We introduced Targeted VFIO Mapping, a novel approach to optimize CVM start-up latency by mapping only essential regions of the shared bounce buffer to the IOMMU, thereby reducing VFIO's overhead. Our evaluation shows that this optimization reduces CVM launch latency by up to 55% on KVM-based platforms for AMD SEV-SNP. By aligning hypervisor memory management with CVM security boundaries, our approach resolves a critical performance bottleneck in practical deployments, while offering a path toward making CVMs a viable alternative for serverless environments.

Open Science. We release the patches to QEMU and Linux, for the prototype used in testing on AMD-SEV for the proposed optimization at: https://github.com/ntu-ssl/start-up-optimization.

Acknowledgements

The authors used generative AI tools solely to assist with writing refinement and stylistic editing. The tools were not used to generate original research data, figures, or technical conclusions. This research was supported by the National Science and Technology Council of Taiwan under research

grants 113-2628-E-002-030-, 114-2628-E-002-001, 114-2634-F-002-003-MBK, and MediaTek Advanced Research Center at National Taiwan University under research grant MTKC-2025-1104.

References

[1] 2020. AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More. https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf.

[2] 2022. Intel® Trust Domain Extensions. https://cdrdv2.intel.com/v1/dl/getContent/690419.

[3] ADLINK Technology, Inc. 2022. AVA Developer Platform Datasheet. https://www.adlinktech.com/Products/Download.ashx?type=MDownload&isDatasheet=yes&file=1967%5cAVA_Developer_Platform-preliminary-datasheet-20220331.pdf

[4] Advanced Micro Devices, Inc. 2023. AMD SEV-TIO: Trusted I/O for Confidential Computing.

[5] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 419–434. https://www.usenix.org/conference/nsdi20/presentation/agache

[6] AMDESE. [n. d.]. qemu. https://github.com/AMDESE/qemu https://github.com/AMDESE/qemu.

[7] AMDESE. 2025. AMD SEV-SNP Linux. https://github.com/AMDESE/linux.

[8] AMDESE. 2025. OVMF. https://github.com/AMDESE/ovmf.

[9] Arm. [n. d.]. *Device Assignment (DA) and Memory Encryption Contexts (MEC)*. https://developer.arm.com/documentation/den0125/0300/Device-Assignment--DA--and-Memory-Encryption-Contexts--MEC-?lang=en.

[10] Arm. 2025. *Learn the architecture - Introducing Arm Confidential Compute Architecture*. https://developer.arm.com/documentation/den0125/0300?lang=en.

[11] Li-Chung Chiang and Shih-Wei Li. 2025. Reload+Reload: Exploiting Cache and Memory Contention Side Channel on AMD SEV. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1014–1027. doi:10.1145/3676641.3716017

[12] Google. 2022. *Protected KVM on arm64: A Technical Deep Dive*. https://osseu2022.sched.com/event/15zBJ/protected-kvm-on-arm64-a-technical-deep-dive-quentin-perret-google.

[13] Google. 2025. crosvm. https://github.com/google/crosvm.

[14] Google. 2025. pKVM Linux. https://android-kvm.googlesource.com/linux/.

[15] Benjamin Holmes, Jason Waterman, and Dan Williams. 2024. SEVeriFast: Minimizing the root of trust for fast startup of SEV microVMs. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 1045–1060. doi:10.1145/3620665.3640424

[16] Dingji Li, Zeyu Mi, Chenhui Ji, Yifan Tan, Binyu Zang, Haibing Guan, and Haibo Chen. 2023. Bifrost: Analysis and Optimization of Network I/O Tax in Confidential Virtual Machines. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 1–15. https://www.usenix.org/conference/atc23/presentation/li-dingji

[17] Mengyuan Li, Yinqian Zhang, Huibo Wang, Kang Li, and Yueqiang Cheng. 2021. CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 717–732. https://www.usenix.org/conference/usenixsecurity21/presentation/li-mengyuan

[18] Shih-Wei Li, John S. Koh, and Jason Nieh. 2019. Protecting Cloud Virtual Machines from Commodity Hypervisor and Host Operating System Exploits. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security 2019)*. Santa Clara, CA, 1357–1374.

[19] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. Formally Verified Memory Protection for a Commodity Multiprocessor Hypervisor. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 3953–3970. https://www.usenix.org/conference/usenixsecurity21/presentation/li-shih-wei

[20] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. A Secure and Formally Verified Linux KVM Hypervisor. In *2021 IEEE Symposium on Security and Privacy (SP)*. 1782–1799.

[21] Don Libes. [n. d.]. *expect(1) - Linux man page*. https://linux.die.net/man/1/expect.

[22] Yunzhuo Liu, Junchen Guo, Bo Jiang, Yang Song, Pengyu Zhang, Rong Wen, Biao Lyu, Shunmin Zhu, and Xinbing Wang. 2025. FastIOV: Fast Startup of Passthrough Network I/O Virtualization for Secure Containers. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 720–735. doi:10.1145/3689031.3696066

[23] Open Infrastructure Foundation. 2024. Kata Containers: The Speed of Containers, the Security of VMs. https://katacontainers.io/.

[24] QEMU Project. 2025. QEMU. https://www.qemu.org/.

[25] Mostafa Saleh. 2025. pKVM SMMUv3 support with trap and emulate. Linux Kernel Mailing List (kvmarm). https://lore.kernel.org/kvmarm/20251117184815.1027271-1-smostafa@google.com/

[26] Benedict Schlüter, Supraja Sridhara, Mark Kuhne, Andrin Bertschi, and Shweta Shinde. 2024. HECKLER: Breaking Confidential VMs with Malicious Interrupts. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 3459–3476. https://www.usenix.org/conference/usenixsecurity24/presentation/schl{ü}ter

[27] The Linux kernel community. 2021. Restricted DMA. https://www.kernel.org/doc/Documentation/devicetree/bindings/reserved-memory/shared-dma-pool.yaml.

[28] The Linux kernel community. 2025. VFIO. https://docs.kernel.org/driver-api/vfio.html.

[29] Michael S. Tsirkin and Cornelia Huck (eds.). 2023. virtio Version 1.3. https://docs.oasis-open.org/virtio/virtio/v1.3/virtio-v1.3.html.